

Topology Survives Shuffling: Robust Long-Context Memory Access Prediction for Graph Analytics

Dongyan Sun^{†*}, Neelesh Gupta^{†*}, Rajgopal Kannan[‡] and Viktor K. Prasanna[†]

[†]University of Southern California, USA

[‡]DEVCOM Army Research Office, USA

Email: {dongyans, neeleshg, prasanna}@usc.edu, rajgopal.kannan.civ@army.mil

Abstract—Graph analytics over biological, social, and cyber-physical networks generate irregular, topology-dependent memory access streams through indirect traversals, limiting the effectiveness of data prefetching. Accurate memory access prediction enables timely data prefetching to hide memory latency. However, machine-learning memory-access predictors couple a short observation history with a fixed prediction horizon across workloads, causing them to fit the node layout of the training trace rather than capture topology-dependent regularities generating the accesses. We propose LoCoMAP, a long-context memory access prediction framework that formulates prediction as a representation learning problem with independently controlled look-back length and look-forward window. We evaluate Single-Source Shortest Paths, PageRank, and Connected Components from the GAP benchmark on six synthetic graph families. Erdős-Rényi (ER) and Grid provide random and deterministic controls, while Barabási-Albert (BA), Copying Model (CM), stochastic block model (SBM), and Watts-Strogatz (WS) capture structure found in biological, web, social, and infrastructure networks, respectively. We test generalization under progressive node shuffling to disrupt memory layout while preserving connectivity. Increasing the history improves F1-score by 8.2% on average across graph-workload pairs and by up to 36.8%, while extending the prediction horizon raises the mean F1-score by 82.3%. Under complete node shuffling, long context improves mean F1-score by 28.9% for ER, BA, CM, and SBM graphs, with a 2× improvement on SBM graphs, but provides no benefit on WS or Grid graphs. Long context remains robust only when node shuffling preserves the range of memory-access deltas, revealing the predictor has learned connectivity structure over node memory layout. This robustness allows a predictor trained on one node layout to remain accurate without retraining, enabling a single prefetcher to generalize across similar networks.

Index Terms—data prefetching, graph analytics, graph topology, memory access prediction, long-context modeling

I. INTRODUCTION

Data prefetching hides memory latency by predicting future memory accesses and fetching data into the cache hierarchy before the processor requests it [1]–[3]. Its effectiveness rests on accurate memory access prediction: correct and timely predictions overlap latency with computation, while incorrect ones waste bandwidth and pollute caches [4]. Rule-based hardware prefetchers exploit simple regularities such as streams and strides but fail when accesses become irregular [5]–[8]. Machine learning (ML) prefetchers instead learn access patterns as sequence models and achieve substantially higher accuracy on irregular workloads [9]–[11], yet graph analytics remain among the hardest applications to predict [12], [13].

Graph analytics underpin web search, recommendation systems, and social networks [14], [15], but they execute through indirect memory accesses (e.g., `row[col[i]]`) whose targets depend on previously loaded values [16], [17], producing sparse access streams with poor locality that leave graph processing memory-bound [12], [18], [19]. This irregularity, however, is not arbitrary [20], [21]: real-world networks exhibit well-understood structure, such as power-law degree distributions [22], [23], small-world connectivity [24], and community organization [25], all of which can be reproduced under controlled conditions by synthetic graph models [22], [24], [26]–[28]. A predictor that captures this structure can use topology as an exploitable prior for memory access prediction.

Existing ML predictors do not exploit this prior. State-of-the-art approaches such as TransFetch [11], SHARP [29], and MPGraph [13] apply a single short-context configuration, with fewer than ten accesses of history, uniformly to all graph workloads, implicitly assuming that all predictive signal lies within a short window and that one configuration fits every graph structure. Neither assumption holds: different topologies induce fundamentally different delta distributions and temporal dependencies. Scale-free graphs [22] revisit hubs at long, irregular intervals; community-structured graphs [25], [28] produce clustered access bursts that exceed any short window; small-world graphs [24] interleave local traversal with long-range jumps; and spatial grids [30] are nearly deterministic. A short-context predictor cannot distinguish these regimes and instead memorizes the vertex layout of its training trace.

Hence, we propose LoCoMAP, a long-context memory access prediction framework for graph analytics. LoCoMAP executes Single-Source Shortest Paths, PageRank, and Connected Components from the GAP benchmark suite [19] on six synthetic graph families: Erdős-Rényi graphs and spatial grids as controls at the extremes of pure randomness and perfect regularity, and Barabási-Albert, Copying Model, stochastic block model, and Watts-Strogatz graphs as representatives of the major structural classes of real-world networks. LoCoMAP formulates prediction as a configurable long-context representation learning problem in which the look-back length L , the look-forward window W , and the delta range D are independent parameters, separating how much history a predictor needs from how far its predictions should reach. Because longer contexts raise inference cost under the strict latency budgets of ML prefetching [10], [31], [32], we characterize the latency

and FLOPs cost of extending context alongside its accuracy gains. Finally, we test whether the learned signal is genuinely topological by evaluating trained predictors under progressively shuffled node orderings, which destroy memory locality while preserving connectivity [33], [34]. We summarize our main contributions as follows:

- We propose LoCoMAP, a long-context memory access prediction framework for graph analytics that maps a long access history to a bitmap of future address deltas, formulating prediction as a representation learning problem with independently controlled look-back length and look-forward window to decouple the history used to learn graph structure from the prediction horizon.
- We develop a topology-controlled evaluation spanning three GAP workloads and six synthetic graph families. Erdős–Rényi and Grid provide random and deterministic endpoint controls, while Barabási–Albert, Copying Model, stochastic block model, and Watts–Strogatz represent structures found in real-world networks. We use progressive node shuffling to separate connectivity-preserving regularities from memory-layout effects and characterize the latency and FLOPs cost of extending context.
- We show that increasing the available history from 8 to 128 accesses improves F1-score by 8.2% on average and by up to 36.8%, while extending the prediction horizon from 8 to 512 accesses raises the overall mean F1-score by 82.3%. Increasing the history across this range increases inference FLOPs by $21\times$ and latency by $6.3\times$.
- We show that under complete node shuffling, without retraining, long context improves mean F1-score by 28.9% across Erdős–Rényi, Barabási–Albert, Copying Model, and stochastic block model graphs, including a $2.05\times$ improvement on stochastic block model graphs, but provides no benefit on Watts–Strogatz or Grid graphs. These results show that long-context robustness emerges when node shuffling preserves the characteristic range of memory-access deltas, so the predictor captures connectivity-dependent regularities beyond the vertex layout in memory.

II. BACKGROUND

A. Graph Analytics and Synthetic Graph Topologies

Graph analytics computes structural properties, such as shortest-path distances, vertex importance, and connectivity, by traversing a graph according to its connectivity [35], [36]. Because traversal order is determined by topology rather than memory layout, the resulting access streams are indirect, with locality that varies across applications and graph structures [12], [16], [19]. Synthetic graph models isolate the influence of topology on memory behavior while avoiding the confounding factors of real-world datasets [20], [37]. As illustrated in Fig. 1, we consider six families, ordered from independent random connectivity to deterministic spatial regularity:

Erdős–Rényi (ER): Erdős–Rényi graphs serve as baseline models for random communication networks and randomized distributed systems [3]. The $G(n, p)$ model includes each

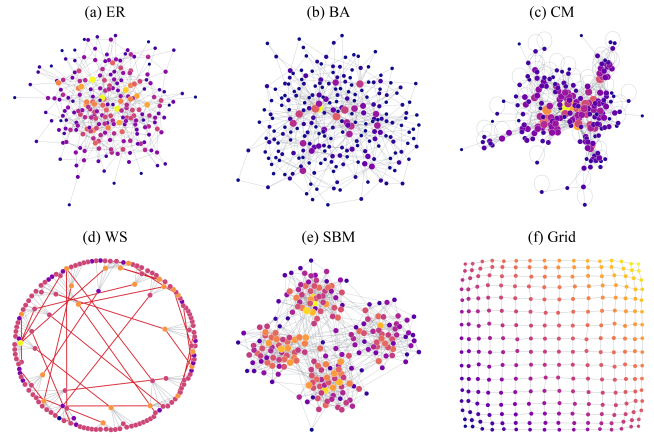


Fig. 1. Graph Topologies of Synthetic Network Models.

possible edge independently with probability p [26], yielding a binomial degree distribution:

$$P(k) = \binom{n-1}{k} p^k (1-p)^{n-1-k} \quad (1)$$

where n is the number of vertices and p is the edge inclusion probability. The result is homogeneous random connectivity with no hubs and no community structure.

Barabási–Albert (BA): Barabási–Albert graphs model scale-free networks with hub-dominated connectivity, including protein interaction networks, citation networks, and Internet autonomous systems [15]. New vertices attach to existing vertices with probability proportional to their degree [22]:

$$\Pi(k_i) = \frac{k_i}{\sum_j k_j} \quad (2)$$

where $\Pi(k_i)$ is the probability that a new vertex connects to vertex i of current degree k_i . Preferential attachment yields a power-law degree distribution $P(k) \propto k^{-3}$ dominated by a small number of high-degree hubs [22].

Copying Model (CM): Copying Model graphs capture the hyperlink structure of the World Wide Web and related information networks, including citation graphs and online document collections [27]. Each arriving vertex selects a random prototype u and forms its i -th out-link either uniformly at random or by copying the prototype’s i -th link:

$$\Pr[v_i = w] = \frac{\alpha}{n} + (1 - \alpha) \mathbb{I}[w = u_i] \quad (3)$$

where v_i is the destination of the new vertex’s i -th link, u_i is the i -th neighbor of the prototype u , n is the current number of vertices, and α is the copying parameter. Copying yields power-law degrees and locally dense link patterns that emerge stochastically rather than from an explicit global partition [27].

Stochastic Block Model (SBM): Stochastic Block Model graphs model community-structured networks such as online social communities, scientific collaboration networks, and protein interaction networks [25]. Vertices are partitioned into K communities by an assignment $z : V \rightarrow \{1, \dots, K\}$,

and each edge is included independently with probability determined by the community memberships of its endpoints:

$$\Pr[(u, v) \in E] = B_{z(u)z(v)} \quad (4)$$

where $\Pr[(u, v) \in E]$ is the probability of an edge between vertices u and v , and $B_{z(u)z(v)}$ is the block connection probability for their communities $z(u)$ and $z(v)$. Setting $B_{ii} > B_{ij}$ for $i \neq j$ produces densely connected clusters joined by sparse inter-community bridges [28], [38].

Watts–Strogatz (WS): Watts–Strogatz graphs model small-world networks, including power grids, neuronal networks, and social networks [24]. Starting from a ring lattice, edges are rewired with probability p ; for intermediate p , the graph combines short paths of a random graph with local clustering:

$$L(p) \gtrsim L_{\text{random}}, \quad C(p) \gg C_{\text{random}} \quad (5)$$

where $L(p)$ and $C(p)$ denote the path length and clustering coefficient at rewiring probability p , and L_{random} and C_{random} the corresponding quantities for an equivalent random graph.

Spatial Grid: Grid graphs represent spatially structured systems such as image pixels, sensor arrays, road networks, and integrated circuit layouts [12]. Vertices lie on a two-dimensional lattice with coordinates (x_u, y_u) and are adjacent exactly at unit Manhattan distance:

$$(u, v) \in E \iff |x_u - x_v| + |y_u - y_v| = 1 \quad (6)$$

where $(u, v) \in E$ denotes an edge between vertices u and v . The result has uniform degree and strong spatial locality [30].

Together, these families span the structural spectrum relevant to our evaluation, from independent randomness to increasingly explicit local and mesoscopic organization [20], [21].

B. Memory Access Prediction

Memory access prediction estimates future memory references from execution history [9], [29]. Following recent ML-based prefetching work [10], [11], we represent each access by its program counter (PC), memory block address, and address delta relative to previous accesses: the PC identifies the referencing instruction, block addresses reduce granularity to cache-line resolution, and deltas encode relative movement that exposes recurring patterns less apparent in absolute addresses [9], [31]. Together, these features provide spatial and program-semantic cues for learning irregular access behavior.

Graph-processing frameworks improve locality through graph partitioning, vertex reordering, and cache-aware data layouts [33], [34], [39]–[41], but largely ignore the access-pattern differences induced by graph topology. We instead use topology-controlled graph families to study whether long context captures topology-induced regularities.

III. RELATED WORK

A. Rule-Based and ML-Based Data Prefetching

Data prefetching hides memory latency by predicting future memory accesses and bringing data into the cache hierarchy before it is requested by the processor [2], [3], [5]. Effective prefetching therefore depends on accurate memory access

prediction [4]. For graph analytics, sparse graph structures and indirect memory references generate irregular access patterns that significantly reduce the effectiveness of conventional hardware prefetchers [12], [17], [42].

1) *Rule-Based Prefetchers:* Existing data prefetching approaches can be broadly divided into rule-based and machine learning (ML)-based methods. Rule-based prefetchers, including stride, stream, and Best-Offset (BO) [6], [8], [43], exploit spatial or temporal locality using predefined heuristics [7], [44] and achieve good performance on regular workloads. However, graph analytics generate sparse and indirect memory accesses that exhibit little predictable locality, limiting the effectiveness of these rule-based designs [12], [19]. To address this challenge, graph-aware prefetchers such as the Indirect Memory Prefetcher (IMP) exploit pointer-based memory references to improve prediction accuracy for graph workloads [45]. However, IMP is primarily designed for L1 cache prefetching and does not address memory access prediction for last-level cache (LLC) prefetching, where accurate prediction matters most: LLC prefetching hides DRAM accesses costing hundreds of cycles, whereas L1/L2 prefetching hides comparatively fast L2/LLC accesses. Consequently, rule-based approaches remain limited in capturing the complex long-context memory access behavior of irregular graph analytics.

2) *ML-Based Prefetchers:* Recently, ML-based prefetchers formulate memory access prediction as a sequence modeling problem [9]. Representative approaches include Voyager [10], TransFetch [11], and MPGraph [13], which learn memory access patterns directly from execution traces instead of relying on fixed heuristics. These methods have demonstrated significantly higher prediction accuracy than traditional prefetchers on irregular workloads [13], [29]. However, their inference latency and storage requirements exceed the strict timeliness constraints of hardware prefetching, especially at the LLC; recent efforts such as Net2Tab narrow this gap by distilling attention-based predictors into compact hierarchies of tables [31], [32].

Existing ML-based approaches, however, fix one short-context model for all workloads and do not examine how topology shapes access behavior. In this work, we treat execution history as a controllable parameter, evaluated across topology-controlled graph families, and use node shuffling to separate learned structure from memorized layout.

IV. APPROACH

LoCoMAP formulates memory access prediction as a configurable long-context sequence-learning problem and evaluates how much predictive information longer context provides across graph topologies.

A. Problem Formulation.

Let $X_t = \{x_1, x_2, \dots, x_L\}$ be the sequence of L history block addresses at time t , where $x_i = a_i \gg \beta$ discards the β -bit block offset from the raw address a_i . Let $P_t = \{pc_1, pc_2, \dots, pc_L\}$ be the sequence of L history program counters at time t , and let $I_t = \{(x_1, pc_1), \dots, (x_L, pc_L)\}$ be the input of the prediction model. Let $O_t = \{o_1, o_2, \dots, o_{2D}\}$

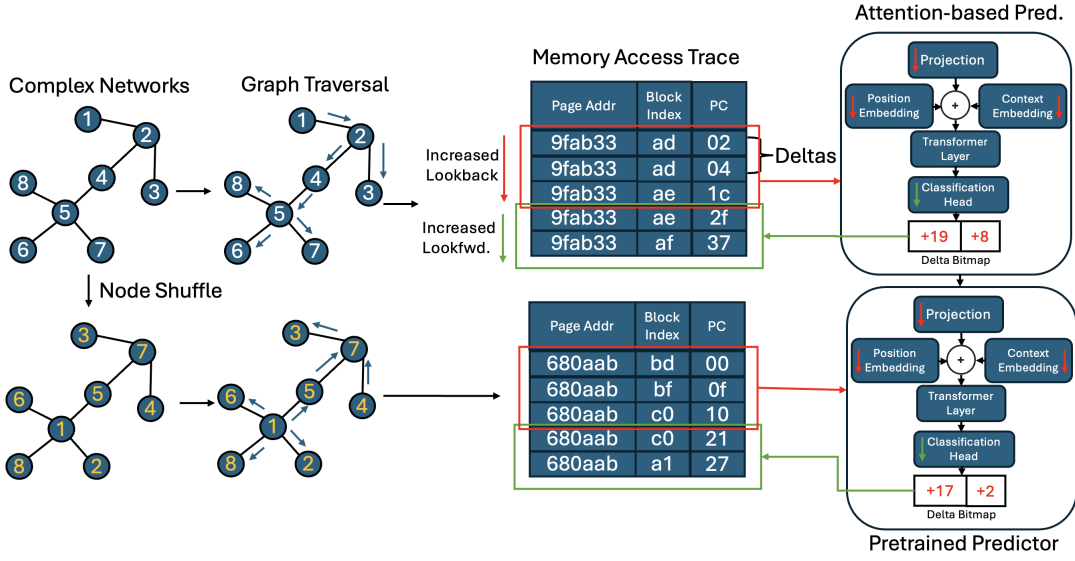


Fig. 2. Overall Architecture of LoCoMAP

be the multi-hot label over the future block deltas within range $[-D, +D]$ occurring across the next W accesses. Our goal is to learn a meaningful mapping from I_t to O_t for data prefetching; final address predictions are the addition of current block address and predicted deltas. Unlike prior work that fixes a short history, we treat look-back length L as a long context to expose long-range dependencies in the access stream.

B. Overall Design

Fig. 2 illustrates the overall workflow of LoCoMAP. A synthetic graph is executed using a GAP workload to generate an LLC memory-access trace. Each memory access is converted to a cache block address and represented using fine-grained address segmentation and execution context. An attention-based predictor takes the previous L accesses as input and predicts a bitmap of future address deltas within a look-forward window W and delta bound D . The predicted deltas are then added to the current block address to recover multiple future memory accesses. The L history block addresses are processed by fine grained address segmentation, and the program counters are hashed and embedded as context. The segmented addresses are mapped by a linear projection, combined with position and context embeddings, and encoded by Transformer layers [46], whose self attention relates every access to the entire L -access history in a single step. A classification head with sigmoid activation outputs the probability p_i of each of the $2D$ deltas, trained with binary cross entropy over the independent labels:

$$\mathcal{L} = -\frac{1}{2D} \sum_{i=1}^{2D} [o_i \log p_i + (1 - o_i) \log(1 - p_i)]. \quad (7)$$

At inference, the output probabilities are binarized with a validation selected threshold that maximizes F1-score, each predicted delta yields the prefetch address x_L plus that delta, so a single inference produces multiple prefetch candidates.

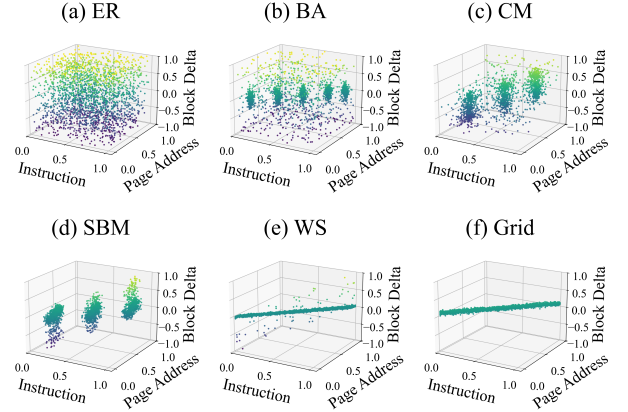


Fig. 3. Visualization of memory access patterns in the benchmark applications. The patterns show diversity in instructions, deltas, and pages, representing various scenarios

V. EVALUATION

A. Experimental Setup

1) *Datasets*: We evaluate LoCoMAP using six representative synthetic graph families that capture common structural properties observed in real-world complex networks as summarized in Table I. The graphs are represented in compressed sparse row (CSR) format for algorithm processing.

TABLE I
GRAPH STATISTICS.

Topology	# Vertices	# Edges	Avg. Degree
ER	1,000,000	2,400,000	4.8
BA	1,000,000	4,999,985	10.0
CM	1,000,000	2,952,366	5.9
SBM	1,000,000	2,500,000	5.0
WS	1,000,000	2,000,000	4.0
Grid	1,000,000	1,998,000	4.0

2) *Algorithms*: We implemented three widely used graph processing algorithms based on GPOP framework [47].

- Single-Source Shortest Paths (SSSP) computes the shortest-path distance from a source vertex to all other vertices by propagating tentative distances along edges until convergence [48].
- PageRank (PR) iteratively computes a vertex-importance score by distributing rank mass from each vertex to its neighbors until the scores converge [35].
- Connected Components (CC) labels each vertex by its component using label propagation, where each vertex adopts the minimum label among its neighbors [49].

TABLE II
BENCHMARK MEMORY TRACE STATISTICS ACROSS GRAPH FAMILIES.

Family	Workload	# Access	# Address	# Page	# Delta
ER	SSSP	14.1K	276	134	1.8K
	PR	4.64M	83.2K	1.5K	507.9K
	CC	4.79M	131.0K	3.2K	613.5K
BA	SSSP	4.50M	117.3K	4.1K	545.1K
	PR	4.81M	64.1K	1.1K	367.3K
	CC	4.81M	61.8K	1.1K	369.8K
CM	SSSP	4.60M	86.9K	3.3K	479.4K
	PR	4.72M	86.2K	1.7K	465.9K
	CC	4.69M	82.1K	1.6K	449.6K
SBM	SSSP	1.12M	23.1K	2.3K	116.6K
	PR	5.19M	85.7K	1.5K	501.7K
	CC	4.84M	101.6K	2.0K	555.4K
WS	SSSP	4.65M	97.2K	3.8K	562.0K
	PR	4.64M	73.8K	1.3K	433.0K
	CC	4.81M	82.3K	1.5K	483.3K
Grid	SSSP	3.53M	25.2K	600	97.2K
	PR	4.75M	74.3K	1.3K	439.3K
	CC	4.82M	68.2K	1.2K	422.9K

B. Metrics

We use F1-score to evaluate memory-access prediction performance. F1-score is the harmonic mean of precision and recall [50], defined as:

$$\text{Precision} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}} \quad (8)$$

$$\text{Recall} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}} \quad (9)$$

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (10)$$

C. Controlled Graph and Trace Generation

We generate all graphs synthetically using the standard generative model of each family, with a fixed target size and comparable average degree, so families differ in structure rather than scale; each graph is stored in CSR format for processing.

To obtain memory-access traces, we develop a Pintool using Intel Pin [51] that detects the memory-access operations executed by each graph workload and records the corresponding context information, including the accessed virtual address and program counter. We then replay the collected accesses through a four-core ChampSim [52] model with a shared last-level cache (LLC) and record the resulting per-core LLC access stream, which forms the input to memory-access prediction.

TABLE III
MODEL CONFIGURATION.

	Configuration	Value	Configuration	Value
Input/output	Delta bound	± 128	Delta bitmap B	256
	History N	9	Look-forward W	256
Attention	Dimension D	64	MLP head layer	1
	Head number	4	Layer L	1

D. Long-Context Configuration and Cost

We investigate long-context memory access prediction by varying the look-back length L and the look-forward window W . Increasing the look-back length L lengthens the input sequence, regularities at any temporal distance within the window, such as hub revisitation and community dwell intervals, become directly reachable by attention. Increasing the look-forward window W aggregates deltas over a longer future horizon, so the bitmap becomes denser and long-range predictive information is converted into additional coverage.

However, The representation gains of long context are not free. Fig. 4 reports inference FLOPs and latency as the look-back length scales from $L=8$ to $L=128$. FLOPs per inference grow from 8M to 169.6M, a $21\times$ increase, and latency grows from 11,357 to 71,345 [cycles], a $6.3\times$ increase, reflecting the quadratic cost of self attention at large L . Setting these costs against Table IV yields the practical tradeoff: for several topology and workload pairs the accuracy gains largely arrive by $L=64$, while cost continues to grow quadratically beyond it, so intermediate context lengths capture much of the benefit at a fraction of the cost. Established deployment techniques for ML prefetchers, including model compression and distance prefetching [13], [31], [32], apply directly to our models and can offset the remaining latency.

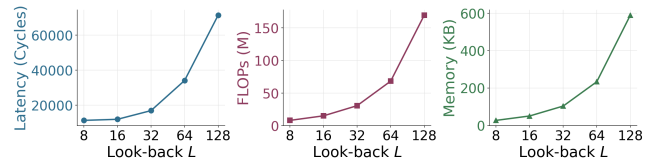


Fig. 4. Latency, FLOPs and memory cost as the look-back length L scales from 8 to 128

E. Training and Evaluation

For each graph–workload pair, we split the collected memory trace into training and testing sets and train a delta predictor to map the access history to a bitmap of future address deltas.

VI. RESULTS

A. Ablation Study

We conduct ablation studies on the two long-context parameters, the look-back length L and the look-forward window W , and report the results in Table IV.

1) *Effectiveness of look-back Length*: Increasing L from 8 to 128 consistently improves prediction accuracy across all workloads. Averaged over the five topologies, the F1-score improves by 5.1% for CC, 3.7% for PR, and 15.7% for SSSP.

TABLE IV
ABLATION OF LOOK-BACK LENGTH L AND LOOK-FORWARD WINDOW W ACROSS GRAPH TOPOLOGIES AND APPLICATIONS (F1-SCORE)

Parameter	Value	Applications																	
		CC						PR						SSSP					
		ER	BA	CM	SBM	WS	Grid	ER	BA	CM	SBM	WS	Grid	ER	BA	CM	SBM	WS	Grid
Look-back L ($W=256, D=256$)	8	0.466	0.561	0.660	0.590	0.690	0.641	0.472	0.658	0.684	0.609	0.733	0.668	0.075	0.256	0.683	0.057	0.363	0.639
	16	0.468	0.574	0.661	0.592	0.691	0.642	0.475	0.669	0.688	0.613	0.738	0.671	0.083	0.273	0.684	0.065	0.354	0.646
	32	0.503	0.568	0.664	0.593	0.714	0.645	0.492	0.661	0.690	0.616	0.740	0.665	0.086	0.303	0.696	0.066	0.365	0.653
	64	0.515	0.620	0.669	0.596	0.731	0.650	0.503	0.670	0.691	0.622	0.748	0.679	0.093	0.311	0.704	0.074	0.358	0.655
Look-forward W ($L=8, D=256$)	128	0.514	0.635	0.674	0.587	0.722	0.648	0.520	0.682	0.693	0.619	0.757	0.681	0.094	0.312	0.716	0.078	0.369	0.663
	8	0.375	0.209	0.369	0.255	0.150	0.263	0.355	0.435	0.372	0.394	0.325	0.373	0.049	0.235	0.594	0.066	0.170	0.537
	32	0.361	0.391	0.486	0.428	0.350	0.402	0.389	0.572	0.464	0.513	0.551	0.527	0.065	0.243	0.600	0.057	0.339	0.621
	128	0.431	0.514	0.589	0.549	0.607	0.581	0.432	0.619	0.601	0.582	0.694	0.605	0.092	0.251	0.648	0.075	0.369	0.625
	256	0.466	0.561	0.660	0.590	0.690	0.641	0.472	0.658	0.684	0.609	0.733	0.668	0.075	0.256	0.683	0.057	0.363	0.639
	512	0.507	0.606	0.710	0.623	0.735	0.696	0.501	0.693	0.736	0.620	0.740	0.713	0.095	0.259	0.682	0.118	0.388	0.654

TABLE V
F1-SCORE ON THE SAME GRAPH UNDER DIFFERENT SHUFFLE PERCENTAGES (ORIGINAL VS. LONG CONTEXT)

Shuffle Percentage	Original Model							Long Context Model						
	ER	BA	CM	SBM	WS	Grid		ER	BA	CM	SBM	WS	Grid	
0%	0.466	0.561	0.660	0.590	0.690	0.641		0.514	0.635	0.674	0.587	0.722	0.648	
25%	0.418	0.467	0.611	0.268	0.318	0.298		0.433	0.510	0.684	0.519	0.325	0.264	
50%	0.436	0.343	0.582	0.402	0.331	0.287		0.476	0.414	0.645	0.563	0.270	0.207	
75%	0.325	0.364	0.430	0.400	0.216	0.261		0.417	0.430	0.454	0.559	0.203	0.136	
100%	0.431	0.350	0.381	0.202	0.218	0.262		0.482	0.406	0.455	0.415	0.185	0.159	

This shows that additional execution history exposes topology-induced regularities that are not visible within a short context window, motivating long-context prediction for graph analytics.

2) *Effectiveness of look-forward Window*: The look-forward window is the dominant factor: extending W from 8 to 512 improves the average F1-score by roughly $2.5\times$ for CC, 74% for PR, and 43% for SSSP. Because a larger output range converts predictive information into additional coverage, this result confirms that much of the exploitable structure in graph access streams lies beyond a short prediction horizon.

B. Generalization Under Node Shuffling

To test whether long context enables the predictor to learn graph topology rather than a particular vertex ordering, we evaluate each trained predictor, without retraining, on CC traces generated after permuting a controlled fraction of vertex identifiers, comparing a short context model ($L=8$) against a long context model ($L=128$); results are shown in Table V.

Fig. 5 shows that shuffling reshapes each family’s delta distribution in a different way: CM and SBM are nearly unchanged; ER sharpens toward small deltas; BA lowers its peak while staying in the same band; and for WS and Grid, the narrow peak of layout induced locality collapses. Because the changes in distribution span from invariant, centralized to decentralized, long context is needed since our delta ranges are wide and differ across families. A short window of eight accesses exposes too few deltas to characterize a family’s range, whereas a long history samples the distribution broadly enough.

Under complete (100%) shuffling, the long context model retains higher accuracy on all four families representing real-world network classes, with the largest gap on community-structured SBM graphs, and ER remains the most robust overall, consistent with its homogeneous random connectivity. These are

exactly the families whose delta distributions survive shuffling in Fig. 5: structure that persists after the layout is destroyed is carried by connectivity, and long context gives the model enough history to exploit it. In contrast, for WS and Grid, whose accuracy derives from layout-induced neighborhood locality, both models collapse and long context confers no advantage. The robustness benefit of long context is therefore specifically a benefit of learning topology structure.

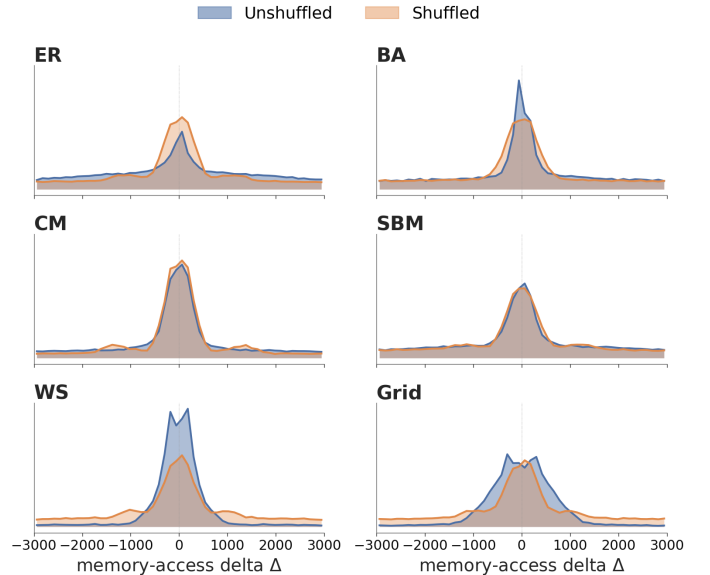


Fig. 5. Range of Delta distributions per graph family, before (blue) and after (orange) node shuffling for Connected Components (CC) workload.



Fig. 6. F1-score under increasing node shuffling ($L=8$ vs. $L=128$, CC workload).

TABLE VI
F1-SCORE RETAINED UNDER FULL NODE SHUFFLING

Graph	Original Model	Long Context Model
ER	92.5%	93.8%
BA	62.4%	63.9%
CM	57.7%	67.5%
SBM	34.2%	70.7%
WS	31.6%	25.6%
Grid	40.9%	24.5%

VII. CONCLUSION

We presented LoCoMAP, a long context memory access prediction framework for graph analytics on complex networks. LoCoMAP treats the look-back length L as a first class parameter, decoupled from the prediction scope set by the look-forward window W , and evaluates this configuration space on three GAP workloads across six topology controlled graph families, with ER and grid graphs as controls and BA, CM, SBM, and WS graphs as representatives of real world network classes. Long context prediction improves F1-score by up to 36.8% over the TransFetch configuration and by 8.2% on average, and extending the prediction horizon raises the overall mean F1-score by 82.3%, while intermediate context lengths capture much of the benefit at a fraction of the quadratic attention cost. Under node shuffling that destroys memory layout while preserving connectivity, long context improves mean F1-score by 28.9% across the four topology carried families, including a $2.05\times$ gain on SBM graphs, and confers no advantage on WS and Grid. The robustness of long context is therefore specifically a benefit of learning topology: a predictor trained on one vertex layout remains accurate without retraining, so a single long context prefetcher can generalize across similarly structured networks rather than being tied

to a particular layout. In future work, we plan to reduce the cost of long context prediction with subquadratic sequence architectures to enable low-latency ML-based prefetchers.

REFERENCES

- [1] M. Dubois, M. Annavaram, and P. Stenström, *Parallel computer organization and design*. Cambridge University Press, 2012.
- [2] S. P. Vanderwiel and D. J. Lilja, "Data prefetch mechanisms," *ACM Computing Surveys (CSUR)*, vol. 32, no. 2, pp. 174–199, 2000.
- [3] S. Byna, Y. Chen, and X.-H. Sun, "A taxonomy of data prefetching mechanisms," in *2008 International Symposium on Parallel Architectures, Algorithms, and Networks (i-span 2008)*. IEEE, 2008, pp. 19–24.
- [4] S. Srinath, O. Mutlu, H. Kim, and Y. N. Patt, "Feedback directed prefetching: Improving the performance and bandwidth-efficiency of hardware prefetchers," in *2007 IEEE 13th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2007, pp. 63–74.
- [5] A. J. Smith, "Sequential program prefetching in memory hierarchies," *Computer*, vol. 11, no. 12, pp. 7–21, 1978.
- [6] N. P. Jouppi, "Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers," *ACM SIGARCH Computer Architecture News*, vol. 18, no. 2SI, pp. 364–373, 1990.
- [7] S. Somogyi, T. F. Wenisch, A. Ailamaki, B. Falsafi, and A. Moshovos, "Spatial memory streaming," *ACM SIGARCH Computer Architecture News*, vol. 34, no. 2, pp. 252–263, 2006.
- [8] P. Michaud, "Best-offset hardware prefetching," in *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2016, pp. 469–480.
- [9] M. Hashemi, K. Swersky, J. R. Smith, G. Ayers, H. Litz, J. Chang, C. Kozyrakis, and P. Ranganathan, "Learning memory access patterns," *arXiv preprint arXiv:1803.02329*, 2018. [Online]. Available: <https://arxiv.org/abs/1803.02329>
- [10] Z. Shi, A. Jain, K. Swersky, M. Hashemi, P. Ranganathan, and C. Lin, "A hierarchical neural model of data prefetching," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 861–873.
- [11] P. Zhang, A. Srivastava, A. V. Nori, R. Kannan, and V. K. Prasanna, "Fine-grained address segmentation for attention-based variable-degree prefetching," in *Proceedings of the 19th ACM International Conference on Computing Frontiers*, ser. CF '22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 103–112. [Online]. Available: <https://doi-org.libproxy1.usc.edu/10.1145/3528416.3530236>
- [12] A. Basak, S. Li, X. Hu, S. M. Oh, X. Xie, L. Zhao, X. Jiang, and Y. Xie, "Analysis and optimization of the memory hierarchy for graph processing workloads," in *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2019, pp. 373–386.
- [13] P. Zhang, R. Kannan, and V. K. Prasanna, "Phases, modalities, spatial and temporal locality: Domain specific ml prefetcher for accelerating graph analytics," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023, pp. 1–15.
- [14] A. Drosou, I. Kalamaras, S. Papadopoulos, and D. Tzovaras, "An enhanced graph analytics platform (gap) providing insight in big network data," *Journal of Innovation in Digital Ecosystems*, vol. 3, no. 2, pp. 83–97, 2016.
- [15] R. Meusel, S. Vigna, O. Lehmborg, and C. Bizer, "The graph structure in the web-analyzed on different aggregation levels," *The Journal of Web Science*, vol. 1, 2015.
- [16] A. Lumsdaine, D. Gregor, B. Hendrickson, and J. Berry, "Challenges in parallel graph processing," *Parallel Processing Letters*, vol. 17, no. 01, pp. 5–20, 2007.
- [17] S. Ainsworth and T. M. Jones, "Graph prefetching using data structure knowledge," in *Proceedings of the 2016 International Conference on Supercomputing*, 2016, pp. 1–11.
- [18] W. A. Wulf and S. A. McKee, "Hitting the memory wall: Implications of the obvious," *ACM SIGARCH Computer Architecture News*, vol. 23, no. 1, pp. 20–24, 1995.
- [19] S. Beamer, K. Asanović, and D. Patterson, "The gap benchmark suite," *arXiv preprint arXiv:1508.03619*, 2017.
- [20] M. E. J. Newman, "The structure and function of complex networks," *SIAM Review*, vol. 45, no. 2, pp. 167–256, 2003.

- [21] R. Albert and A.-L. Barabási, “Statistical mechanics of complex networks,” *Reviews of Modern Physics*, vol. 74, no. 1, pp. 47–97, 2002.
- [22] A.-L. Barabási and R. Albert, “Emergence of scaling in random networks,” *Science*, vol. 286, no. 5439, pp. 509–512, 1999.
- [23] A. Broder, R. Kumar, F. Maghoul, P. Raghavan, S. Rajagopalan, R. Stata, A. Tomkins, and J. Wiener, “Graph structure in the web,” *Computer Networks*, vol. 33, no. 1–6, pp. 309–320, 2000.
- [24] D. J. Watts and S. H. Strogatz, “Collective dynamics of ‘small-world’ networks,” *Nature*, vol. 393, no. 6684, pp. 440–442, 1998.
- [25] M. Girvan and M. E. J. Newman, “Community structure in social and biological networks,” *Proceedings of the National Academy of Sciences*, vol. 99, no. 12, pp. 7821–7826, 2002.
- [26] P. Erdős and A. Rényi, “On random graphs. i,” *Publicationes Mathematicae Debrecen*, vol. 6, no. 3–4, pp. 290–297, 1959.
- [27] R. Kumar, P. Raghavan, S. Rajagopalan, D. Sivakumar, A. Tomkins, and E. Upfal, “Stochastic models for the web graph,” in *Proceedings of the 41st Annual Symposium on Foundations of Computer Science (FOCS)*, 2000, pp. 57–65.
- [28] E. Abbe, “Community detection and stochastic block models: recent developments,” *The Journal of Machine Learning Research*, vol. 18, no. 1, pp. 6446–6531, 2017. [Online]. Available: <https://arxiv.org/abs/1703.10146>
- [29] P. Zhang, R. Kannan, X. Tong, A. V. Nori, and V. K. Prasanna, “Sharp: Software hint-assisted memory access prediction for graph analytics,” in *2022 IEEE High Performance Extreme Computing Conference (HPEC)*, 2022, pp. 1–8.
- [30] M. Barthélemy, “Spatial networks,” *Physics Reports*, vol. 499, no. 1–3, pp. 1–101, 2011.
- [31] P. Zhang, N. Gupta, R. Kannan, and V. K. Prasanna, “Attention, distillation, and tabularization: Towards practical neural network-based prefetching,” *arXiv preprint arXiv:2401.06362*, 2024.
- [32] —, “Net2Tab: Tabularizing neural networks with applications to data prefetching,” *Journal of Parallel and Distributed Computing*, 2026, to appear.
- [33] H. Wei, J. X. Yu, C. Lu, and X. Lin, “Speedup graph processing by graph ordering,” in *Proceedings of the 2016 International Conference on Management of Data (SIGMOD)*, 2016, pp. 1803–1818.
- [34] V. Balaji and B. Lucia, “When is graph reordering an optimization? Studying the effect of lightweight graph reordering,” in *2018 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2018, pp. 203–214.
- [35] L. Page, S. Brin, R. Motwani, and T. Winograd, “The PageRank citation ranking: Bringing order to the web,” Stanford InfoLab, Tech. Rep. 1999-66, 1999.
- [36] G. Malewicz, M. H. Austern, A. J. C. Bik, J. C. Dehnert, I. Horn, N. Leiser, and G. Czajkowski, “Pregel: A system for large-scale graph processing,” in *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data*, 2010, pp. 135–146.
- [37] D. Chakrabarti, Y. Zhan, and C. Faloutsos, “R-MAT: A recursive model for graph mining,” in *Proceedings of the 2004 SIAM International Conference on Data Mining (SDM)*, 2004, pp. 442–446.
- [38] P. W. Holland, K. B. Laskey, and S. Leinhardt, “Stochastic blockmodels: First steps,” *Social Networks*, vol. 5, no. 2, pp. 109–137, 1983.
- [39] J. E. Gonzalez, Y. Low, H. Gu, D. Bickson, and C. Guestrin, “Powergraph: Distributed graph-parallel computation on natural graphs,” in *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*, 2012, pp. 17–30.
- [40] J. Shun and G. E. Blelloch, “Ligra: A lightweight graph processing framework for shared memory,” in *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2013, pp. 135–146.
- [41] A. Kyröla, G. Blelloch, and C. Guestrin, “Graphchi: Large-scale graph computation on just a pc,” in *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*, 2012, pp. 31–46.
- [42] S. Ainsworth and T. M. Jones, “Software prefetching for indirect memory accesses: A microarchitectural perspective,” *ACM Transactions on Computer Systems (TOCS)*, vol. 36, no. 3, pp. 1–34, 2018.
- [43] T.-F. Chen and J.-L. Baer, “Effective hardware-based data prefetching for high-performance processors,” *IEEE Transactions on Computers*, vol. 44, no. 5, pp. 609–623, 1995.
- [44] A. Jain and C. Lin, “Linearizing irregular memory accesses for improved correlated prefetching,” in *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2013, pp. 247–259.
- [45] J. Yu, H. Lu, C. Ding, and C. Dwyer, “Imp: Indirect memory prefetcher,” in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*, 2017, pp. 178–190.
- [46] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [47] K. Lakhotia, R. Kannan, S. Pati, and V. Prasanna, “Gpop: A scalable cache-and memory-efficient framework for graph processing over parts,” *ACM Transactions on Parallel Computing (TOPC)*, vol. 7, no. 1, pp. 1–24, 2020.
- [48] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [49] Y. Shiloach and U. Vishkin, “An $o(\log n)$ parallel connectivity algorithm,” *Journal of Algorithms*, vol. 3, no. 1, pp. 57–67, 1982.
- [50] D. M. W. Powers, “Evaluation: from precision, recall and f-measure to roc, informedness, markedness and correlation,” *arXiv preprint arXiv:2010.16061*, 2020. [Online]. Available: <https://arxiv.org/abs/2010.16061>
- [51] C.-K. Luk, R. Cohn, R. Muth, H. Patil, A. Klauser, G. Lowney, S. Wallace, V. J. Reddi, and K. Hazelwood, “Pin: Building customized program analysis tools with dynamic instrumentation,” in *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2005, pp. 190–200.
- [52] N. Gober, G. Chacon, L. Wang, P. V. Gratz, D. A. Jiménez, E. Teran, S. Pugsley, and J. Kim, “The championship simulator: Architectural simulation for education and competition,” 2022. [Online]. Available: <https://arxiv.org/abs/2210.14324>